

EduPlatform

Платформа для онлайн-курсов

Описание

Необходимо разработать backend-систему для платформы онлайн-обучения, состоящую из **двух сервисов**. Платформа позволяет преподавателям создавать курсы с уроками, а студентам — записываться на них и просматривать материалы. Загружаемые изображения (обложки курсов, иллюстрации к урокам) проходят автоматическую обработку: сжатие и наложение водяного знака.

Архитектура: два сервиса

Сервис 1 — Main API (Nest.js)

Основной сервер, отвечающий за всю бизнес-логику:

- **Авторизация и регистрация** с использованием JWT-токенов
- **Две роли:** `student` и `teacher`
- **CRUD курсов** (создание, просмотр, редактирование, удаление) — доступно только преподавателям-владельцам
- **CRUD уроков** внутри курсов
- **Запись студента на курс**
- **Загрузка изображений** (обложки курсов и картинки к урокам) с последующей отправкой события в Kafka
- **Кэширование** списка курсов и детальной информации о курсе через Redis с инвалидацией кэша при изменении данных

Сервис 2 — Image Worker (Express.js)

Отдельный микросервис для обработки изображений:

- Слушает топик Kafka и получает задачи на обработку
- Сжимает изображение до разумного размера и качества (библиотека Sharp)

- Накладывает водяной знак (watermark)
 - Сохраняет обработанный файл
 - Обновляет статус изображения в базе данных (например, `processing` → `ready`)
 - После обработки отправляет событие в Kafka о завершении
-

Технологический стек

Технология	Где и зачем используется
Nest.js	Main API — основная бизнес-логика
Express.js	Image Worker — микросервис обработки изображений
MongoDB	Основная база данных
Mongoose	ODM для работы с MongoDB (схемы, валидация, связи)
Redis	Кэширование данных о курсах с инвалидацией
Kafka	Асинхронное взаимодействие между сервисами (два топика: отправка задачи на обработку и уведомление о завершении)
Sharp	Сжатие изображений и наложение водяного знака

Модели данных

User

Имя, email (уникальный), хэш пароля, роль (`student` / `teacher`), список курсов, на которые записан студент

Course

Название, описание, ссылка на преподавателя, обложка (URL + статус обработки), список уроков, количество записавшихся студентов

Lesson

Название, текстовый контент, ссылка на курс, прикрепленные изображения (URL + статус обработки), порядковый номер

Требуемые API-эндпоинты

Авторизация

- Регистрация (с указанием роли)
- Логин (возвращает JWT-токен)

Курсы

- Получить список всех курсов (*кэшируется в Redis*)
- Получить детали курса по ID (*кэшируется в Redis*)
- Создать курс (*только teacher*)
- Обновить курс (*только teacher-владелец, инвалидирует кэш*)
- Удалить курс (*только teacher-владелец, инвалидирует кэш*)
- Записаться на курс (*только student*)

Уроки

- Получить все уроки курса
- Добавить урок (*только teacher-владелец курса*)
- Обновить урок
- Удалить урок

Загрузка изображений

- Загрузить обложку курса
 - Загрузить изображение к уроку
 - Получить обработанное изображение по имени файла
-

Поток обработки изображений

1. Преподаватель загружает картинку через Main API
 2. Файл сохраняется в папку с оригиналами, в базе статус — `processing`
 3. Main API отправляет сообщение в Kafka (топик `image.uploaded`) с информацией о файле
 4. Image Worker получает сообщение, обрабатывает картинку (сжатие + водяной знак)
 5. Обработанный файл сохраняется в отдельную папку
 6. Image Worker обновляет статус в MongoDB на `ready`
 7. Image Worker отправляет сообщение в Kafka (топик `image.processed`) — подтверждение
-

Инфраструктура

Все внешние зависимости (MongoDB, Redis, Kafka + Zookeeper) должны подниматься через `docker-compose.yml` одной командой.

Что нужно сдать

1. Исходный код обоих сервисов
 2. `docker-compose.yml` для запуска инфраструктуры
 3. `README.md` с инструкцией:
 - как запустить проект
 - как протестировать основные сценарии
 - описание архитектуры и принятых решений
-

Критерии оценки

Критерий	Баллы
Авторизация (JWT + разделение по ролям)	10

Критерий	Баллы
CRUD курсов и уроков (Nest.js + Mongoose)	20
Кэширование в Redis (с инвалидацией при изменениях)	15
Kafka (producer + consumer, два топика)	20
Image Worker на Express (сжатие + водяной знак)	20
Docker Compose (всё поднимается одной командой)	10
README с инструкцией запуска	5
Итого	100

Рекомендуемый план работы

Период	Что делать
Март, недели 1–2	Настройка проекта, docker-compose, подключение MongoDB, описание Mongoose-схем, модуль авторизации
Март, недели 3–4	CRUD курсов и уроков в Nest.js, Guards для ролей
Апрель, недели 5–6	Подключение Redis, кэширование списка и деталей курсов, инвалидация
Апрель, недели 7–8	Настройка Kafka, создание Image Worker на Express, обработка изображений через Sharp
Май, недели 9–10	Эндпоинт загрузки файлов, полная интеграция: загрузка → Kafka → обработка → обновление статуса
Май, недели 11–12	Тестирование, исправление багов, написание README
Июнь	Защита проекта